



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

∂ CBDs: Differentiable Causal Block Diagrams

Ján Drgoňa

Associate Professor, Civil and Systems Engineering Department

Electrical and Computer Engineering Department (secondary)

The Ralph O'Connor Sustainable Energy Institute (ROSEI)

Data Science and AI Institute (DSAI)

ICCPS Conference, Saint Malo, France

Co-Authors of ∂ CBDs: Differentiable Causal Block Diagrams



Thomas Beckers



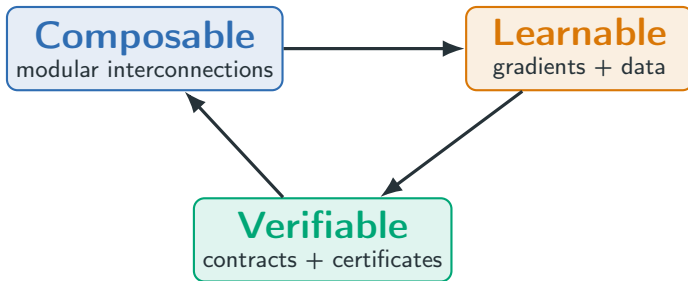
Ján Drgoňa



Truong X. Nghiem



Motivation: Next Generation Cyber-Physical Systems (CPS)



Modern CPS need the integration of composability, verifiability, and learning

Existing frameworks and modeling languages usually optimize one or two properties, but rarely provide all three in a single executable modeling pipeline.

Language 1: Causal Block Diagrams (CBD)

Formal object

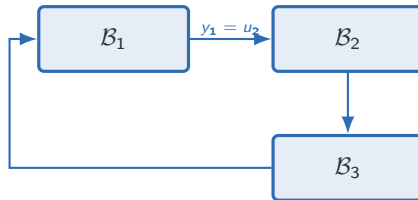
$$\mathcal{B} = (U, X, Y, T, F, g, \text{Init})$$

Block is defined by inputs, states, outputs, time semantics, state transitions, output map, and initial condition.

Closure under composition

$$\mathcal{D} = (\{\mathcal{B}_i\}_{i=1}^N, \Sigma)$$

A CBD is a collection of causal blocks $\mathcal{B}_i$ connected by typed signal wires Σ . Each block maps input signals to output signals through state and output semantics, satisfying causality.



Composable by construction: serial, parallel, and feedback composition preserve interface, state, and time semantics, allowing hierarchical subsystems to be collapsed into a single equivalent block.

Language 2: Differentiable Programs

Formal object

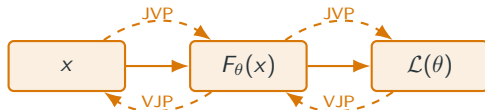
$$F_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \mathcal{L}(\theta) = \ell(F_\theta(x), y)$$

A differentiable program is an executable computational graph whose intermediate operations expose derivative information.

Local derivative actions

$$\text{JVP}(v) = J_F v, \quad \text{VJP}(\bar{v}) = J_F^\top \bar{v}$$

AD propagates local Jacobian actions rather than forming the full Jacobian.



Composable by chain rule: elementary differentiable operators compose into larger computational graphs, and local Jacobian actions enable efficient end-to-end gradient propagation.

However, the semantics are primarily numerical, with limited native support for modular correctness, contracts, or formal verification.

Language 3: Assume–Guarantee Contracts

Formal object

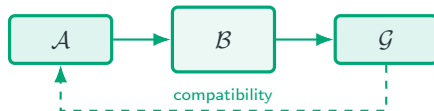
$$\mathcal{C}^{\text{AG}} = (\mathcal{A}, \mathcal{G})$$

The environment must satisfy assumptions $\mathcal{A}$; the component $\mathcal{B}$ then guarantees behavior in $\mathcal{G}$.

Modular Reasoning

$$U \in \mathcal{A} \implies (U, \mathcal{B}(U)) \in \mathcal{G}$$

Contracts support modular reasoning: upstream guarantees must satisfy downstream assumptions. Local contracts compose hierarchically, allowing system-level correctness to be derived from component-level specifications.



Composable by refinement: local assumptions and guarantees compose across interfaces, enabling scalable hierarchical verification and abstraction.

However, contract reasoning is traditionally separated from differentiable optimization and data-driven learning.

The Gap: Three Powerful Languages, Weakly Connected

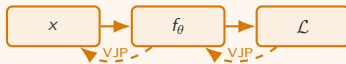
CBDs



Pros: Great for modular CPS design.

Cons: Not differentiable; hard to optimize.

Differentiable programs



Pros: Great for optimization and learning.

Cons: Limited semantic structure for correctness and reasoning.

Contracts



Pros: Great for modular reasoning.

Cons: Disconnected from learning.

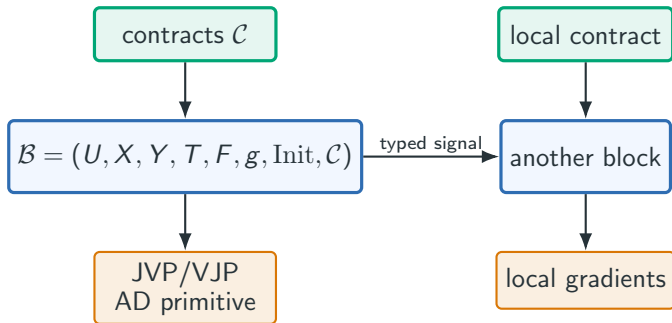
Question:

Can a CPS diagram be simultaneously a causal model, a differentiable program, and a contract-carrying object?

Our answer: Differentiable Causal Block Diagrams (∂ CBDs)

Informal Definition

A ∂ CBD is a causal block diagram whose blocks expose causal execution semantics, contract semantics, and local derivative actions.



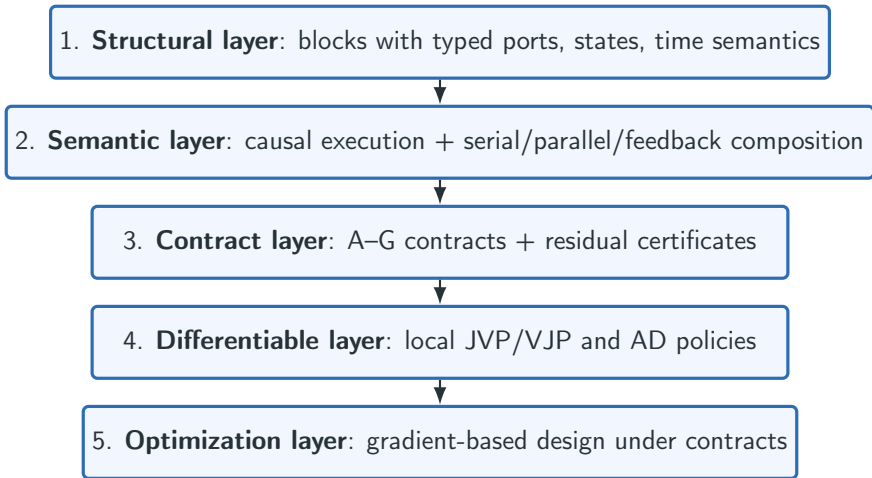
Contracts guide learning; gradients make contracts trainable; CBDs preserve modular structure.

Position paper presenting vision for next generation semantics for CPS:

1. **Differentiable CBD formalism:** blocks preserve causal execution semantics while exposing AD-compatible derivative operations.
2. **Contract integration:** assume–guarantee contracts for semantic correctness and residual contracts for differentiable certificates.
3. **AD-ready reduction:** rate lifting, branch expansion, feedback unrolling, and topological sorting compile ∂ CBDs to finite-horizon DAGs.
4. **Verifiable optimization layer:** objectives and contract residuals are optimized jointly by gradient-based methods.

∂ CBDs turn block diagrams into differentiable programs with modular composition and verification.

Five-layer abstraction of ∂ CBDs

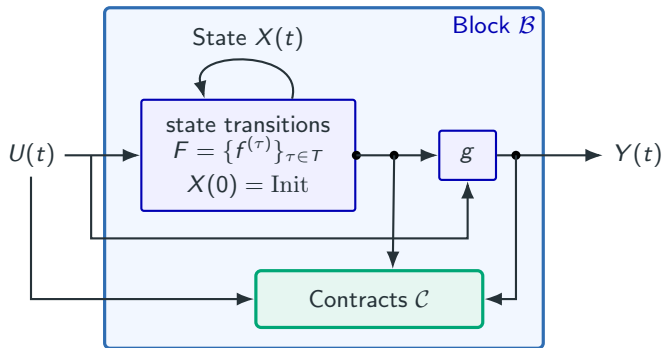


Each layer adds structure without discarding the semantics of the previous layer.

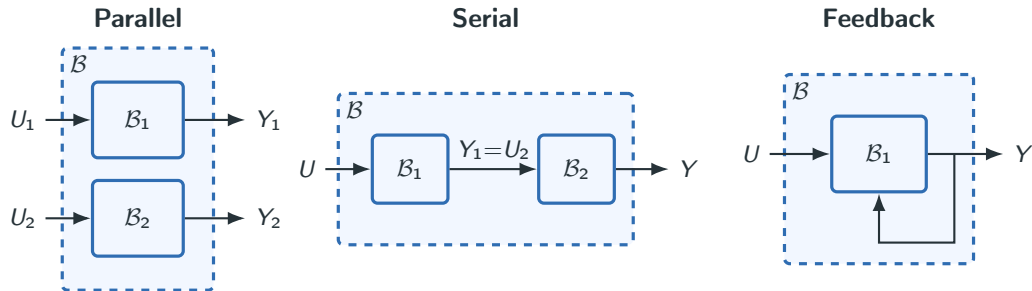
Block with a contract

$$\mathcal{B} = (U, X, Y, T, F, g, \text{Init}, \mathcal{C})$$

- U, X, Y : input, state, output signals
- T : continuous/discrete/hybrid time semantics
- F : state transitions or dynamics
- g : output map or implicit solver
- Init : initial conditions
- $\mathcal{C}$: contract annotations



The same object is used for simulation, verification, and differentiation.



Key property

Composition rules preserve CBD structure: Algebraic-loop-free CBDs can be flattened into a directed acyclic graph (DAG) of blocks; nested subsystems may be represented by heterogeneous blocks while preserving interface semantics.

Contract layer

A–G contracts

$$\mathcal{C}^{\text{AG}} = (\mathcal{A}, \mathcal{G}),$$

$$U \in \mathcal{A} \Rightarrow (U, \mathcal{B}(U)) \in \mathcal{G}$$

System-wide quantitative certificates
Enable modular correctness reasoning

$$\mathcal{C}_{\parallel}^{\text{res}} = \begin{bmatrix} \mathcal{C}_1^{\text{res}}(X_1, U_1, Y_1) \\ \mathcal{C}_2^{\text{res}}(X_2, U_2, Y_2) \end{bmatrix} \leq 0$$

$$\mathcal{C}_{; }^{\text{res}} = \begin{bmatrix} \mathcal{C}_1^{\text{res}}(X_1, U_1, Y_1) \\ \mathcal{C}_2^{\text{res}}(X_2, (U_2^-, Y_1^\sigma), Y_2) \end{bmatrix} \leq 0$$

$$\mathcal{C}_{\circlearrowleft}^{\text{res}} = \mathcal{C}^{\text{res}}(X, (U^-, Y^\sigma), Y) \leq 0$$

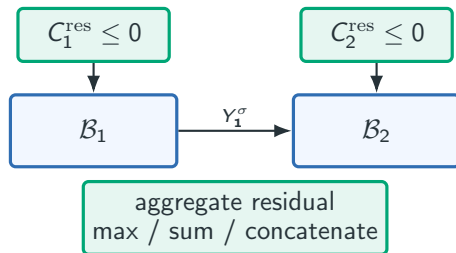
Residual contracts

$$\mathcal{C}^{\text{res}}(X, U, Y; \theta) \leq 0$$

Quantitative certificates on executions

Differentiable and AD-compatible

Compatible with physics-informed machine learning



Residual contracts bridge formal specifications and numerical optimization, and follow the same signal substitutions as the CBD execution semantics.

Almost-everywhere differentiability assumption

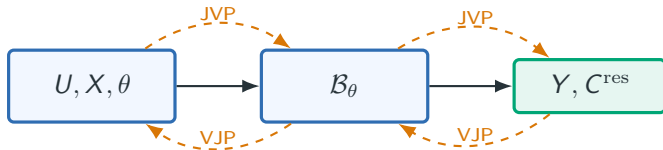
State transitions $f^{(\tau)}$, output maps g , and residuals $\mathcal{C}^{\text{res}}$ are locally Lipschitz and differentiable almost everywhere, with a declared policy at isolated nonsmooth points.

$$\text{JVP}(v) = J_F v$$

Forward sensitivity propagation via
Jacobian-vector product (JVP)

$$\text{VJP}(\bar{v}) = J_F^\top \bar{v}$$

Reverse-mode adjoint propagation via
Vector-Jacobian product (VJP).



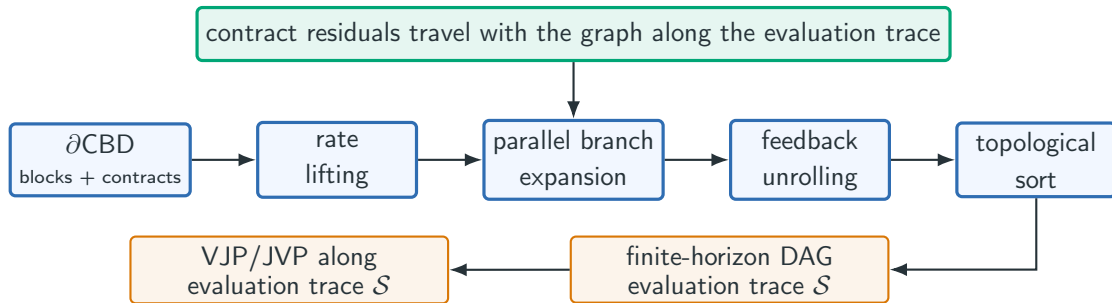
Differentiable blocks expose local JVP/VJP operations.

Differentiable layer: Heterogeneous blocks with AD primitives

Block type	AD primitive
Discrete-time dynamics	Unroll dynamics; backpropagation through time (BPTT)
Continuous-time dynamics	Adjoint state methods
Neural components	Native automatic differentiation through layers via VJP
Fixed-point / implicit blocks	Differentiate via the implicit function theorem (IFT)
Optimization layers	Differentiate KKT systems via IFT
Integer / logic ports	Declared relaxation, estimator, or surrogate-gradient policy
Control flow	AD over realized evaluation trace

The ∂ CBD framework can mix physical simulators, neural networks, controllers, implicit solvers, and optimization layers in one contract-aware computational graph.

Differentiable layer: Reduction to an AD-ready DAG



AD applies local Jacobian actions along the evaluation trace.

Differentiable Causal Block Diagrams (∂ CBD)

Causal Block Diagram (CBD)

A CBD is a composition of blocks $\mathcal{D} = (\mathcal{B}, \Sigma)$, $\mathcal{B} = \{\mathcal{B}_i\}_{i=1}^N$ with unilateral typed signal connections $\Sigma = \{\sigma_{i,j} \mid \mathcal{B}_i \in \mathcal{B}, \mathcal{B}_j \in \mathcal{B}\}$. Its execution defines the composite map

$$(X, Y) \in \text{Exec}(\mathcal{D}, U).$$

Serial, parallel, and feedback composition preserve interface, state, and time semantics.

Differentiable CBD (∂ CBD)

The CBD $\mathcal{D}$ becomes *differentiable* if:

1. each $\mathcal{B}_i$ exposes local JVP/VJP rules;
2. Σ is type-consistent and feedback-admissible;
3. the execution map $\text{Exec}(\mathcal{D}, U)$ is differentiable a.e.

Composable model $\Rightarrow$ differentiable execution $\Rightarrow$ optimization

General ∂ CBD optimization problem

General ∂ CBD Optimization Problem

$$\min_{\theta} \mathcal{J}(Y(t), U(t), X(t); \theta) + w_{\text{reg}} \mathcal{R}(\theta)$$

$$\text{s.t. } (X(t), Y(t)) \in \text{Exec}(\mathcal{D}(\theta), U(t)), \quad \text{where}$$

$$\text{dynamics: } \begin{cases} \dot{X}^{(0)}(t) = f^{(0)}(X(t), U(t); \theta), & t \in \mathcal{T}, \\ X_{k+1}^{(\tau)} = f^{(\tau)}(X_k, U_k; \theta), & \tau > 0, k \in \mathcal{T}, \end{cases}$$

$$\text{outputs: } Y(t) = g(X(t), U(t); \theta),$$

$$\text{contracts: } \mathcal{C}^{\text{res}}(X(t), U(t), Y(t); \theta) \leq 0, \quad \forall t \in \mathcal{T}.$$

AD-compatible relaxation

$$\min_{\theta} \mathcal{L}(\theta) = \mathcal{J} + w_{\text{reg}} \mathcal{R} + w_{\mathcal{C}} \Phi_{\beta}(\mathcal{C}^{\text{res}})$$

This general formulation covers constrained system identification, policy learning, control co-design, physics-informed machine learning, and differentiable optimization workflows.

Three examples: same ∂ CBD recipe, different guarantees

Example	Contract	Optimization mechanism
1: Scalar feedback gain	Input-to-state stability (ISS)	Projected gradient onto stable interval
2: Neural policy with Lyapunov certificate	Trajectory-level Lyapunov decrease	Penalty on residual
3: Stable deep Koopman system identification	Latent operator stability $\ K\ _2 < 1$	Stability-by-construction via parameterization of K



Example 1: Stability-by-construction gain tuning

Closed-loop scalar plant

$$\mathcal{B}_{\text{cl}} = \circ(\mathcal{B}_{\Sigma}; \mathcal{B}_C; \mathcal{B}_P)$$

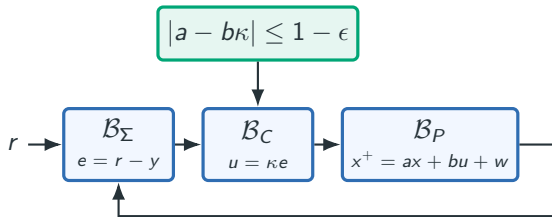
$$\mathcal{B}_P: x_{k+1} = ax_k + bu_k + w_k, y_k = x_k$$

$$\mathcal{B}_C: u_k = \kappa(e_k)$$

$$\mathcal{B}_{\Sigma}: e_k = r_k - y_k$$

Stability residual

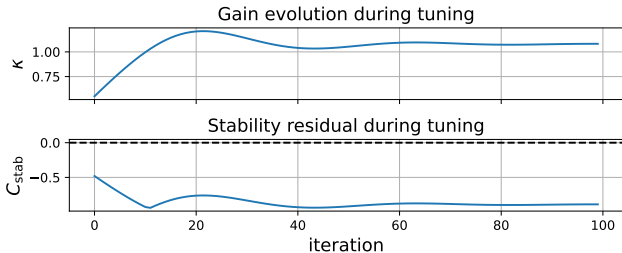
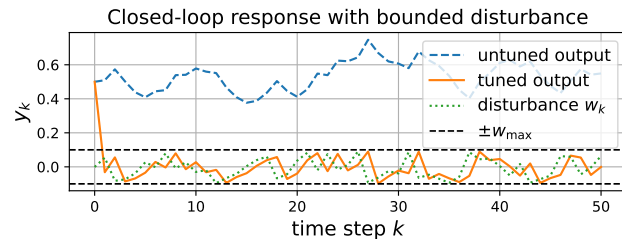
$$\mathcal{C}_{\text{stab}}^{\text{res}}(\kappa, a, b) = |a - b\kappa| - (1 - \epsilon) \leq 0$$



This is a **parameter-level certificate**: stability is built into the closed-loop system dynamics.

$$\begin{aligned} \min_{\kappa \in \mathbb{R}} \quad & \mathcal{J}(\kappa) := \sum_{k=0}^N (\|y_k\|^2 + \lambda_u \|u_k\|^2), \\ \text{s.t.} \quad & (U, Y) \in \text{Exec}(\mathcal{B}_{\text{cl}}(\kappa), W), \\ & \mathcal{C}_{\text{stab}}^{\text{res}}(\kappa, a, b) \leq 0. \end{aligned}$$

Example 1: Stability-by-construction gain tuning



Training improves performance while preserving the stability contract $C_{\text{stab}}^{\text{res}} \leq 0$.

Example 2: Joint neural policy and Lyapunov certificate

Closed-loop CBD

$$\mathcal{B}_{\text{cl}}^{\text{vdp}} = \circ (\mathcal{B}_{\pi}; \mathcal{B}_P; \mathcal{B}_V)$$

$$\mathcal{B}_{\pi} : u_k = \pi_{\Theta}(x_k)$$

$$\mathcal{B}_P : x_{k+1} = f(x_k, u_k)$$

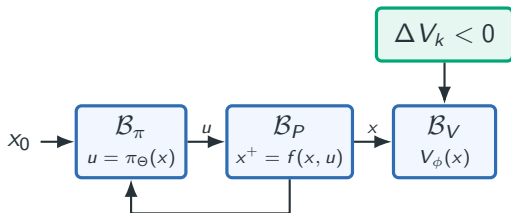
$$\mathcal{B}_V : V_{\phi}(x_k)$$

$$\theta = (\Theta, \phi)$$

Lyapunov residual

$$\mathcal{C}_{\text{lyap}}^{\text{res}}(X; \phi) =$$

$$\max_k [V_{\phi}(x_{k+1}) - V_{\phi}(x_k) + \varepsilon] \leq 0$$



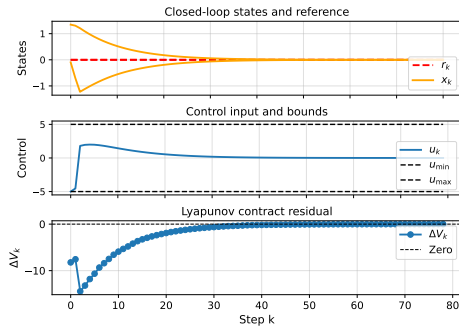
The residual is a **trajectory-level certificate**: it certifies the realized rollout.

$$\min_{\theta} \quad \mathcal{J}(\theta) := \sum_{k \in \mathcal{T}} \|x_k\|^2$$

$$\text{s.t.} \quad (X, U, V) \in \text{Exec}(\mathcal{B}_{\text{cl}}^{\text{vdp}}(\theta), x_0),$$

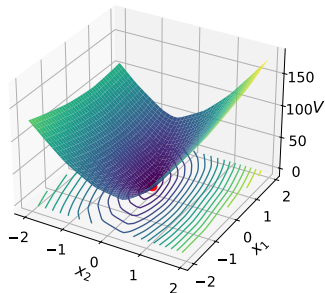
$$\mathcal{C}_{\text{lyap}}(X; \phi) \leq 0, \quad u_{\min} \leq u_k \leq u_{\max}, \quad \forall k \in \mathcal{T}.$$

Example 2: Joint neural policy and Lyapunov certificate



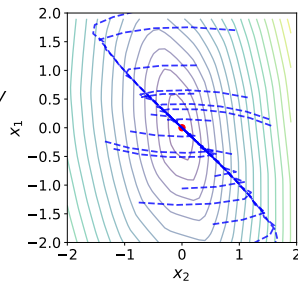
Closed-loop rollout

neural control with Lyapunov contract



Learned certificate

Lyapunov landscape and level sets



Joint optimization of policy π_{Θ} and certificate V_{ϕ} produces stable closed-loop trajectories satisfying the trajectory-wise residual contract $\mathcal{C}_{\text{lyap}}^{\text{res}} \leq 0$.

Example 3: Stable-by-construction Deep Koopman identification

Koopman CBD

$$\mathcal{B}_{\text{koop}}(\theta) = (\mathcal{B}_{\text{enc}}(\theta_1); \mathcal{B}_K(\theta_2); \mathcal{B}_{\text{dec}}(\theta_3))$$

$$\mathcal{B}_{\text{enc}}(\theta_1): z_k = \text{NN}_{\theta_1}(y_k)$$

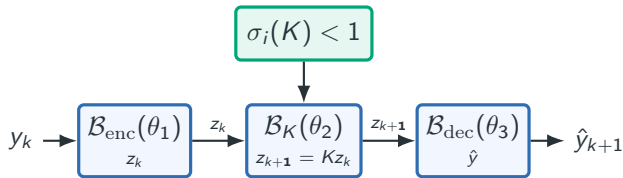
$$\mathcal{B}_K(\theta_2): z_{k+1} = K(\theta_2)z_k$$

$$\mathcal{B}_{\text{dec}}(\theta_3): \hat{y}_k = \text{NN}_{\theta_3}(z_k)$$

Stability contract

$$\mathcal{C}_{\text{stab}}(K) = \|K\|_2 - 1 \leq 0$$

Enforced by bounded singular-value parameterization.

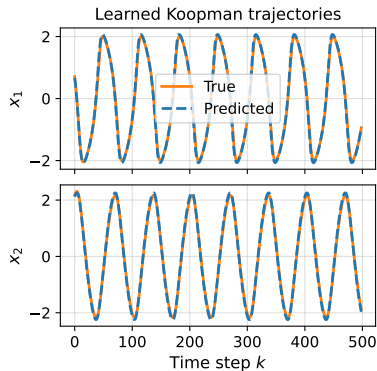


This is a **parameter-level certificate**: stability is built into the latent dynamics.

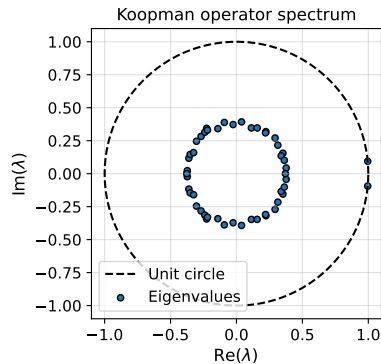
$$\min_{\theta} \mathcal{J}(\theta) := \sum_{i=1}^m (\ell_y^i + \ell_{\text{lin}}^i + \ell_{\text{recon}}^i)$$

$$\text{s.t. } (X^i, Y^i) \in \text{Exec}\left(\mathcal{B}_{\text{koop}}(\theta), \hat{Y}_0^i\right), \quad i = 1, \dots, m, \\ \mathcal{C}_{\text{stab}}(K(\theta_2)) \leq 0.$$

Example 3: Stable-by-construction Deep Koopman identification



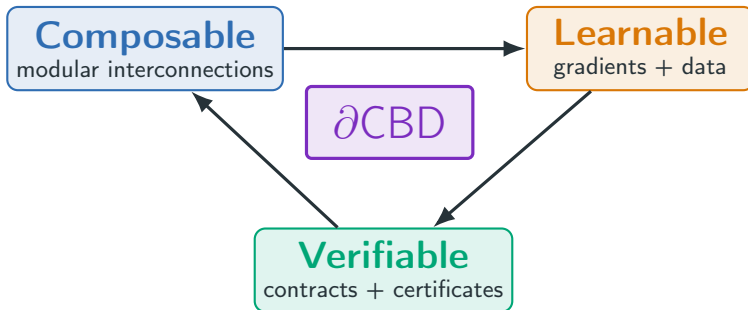
Prediction accuracy
true vs. Koopman rollouts



Stability certificate
spectrum of the learned Koopman operator

The learned Koopman model accurately predicts nonlinear dynamics while enforcing the parameter-level contract $\mathcal{C}_{\text{stab}}(K) \leq 0$, ensuring all eigenvalues remain inside the unit disk.

Takeaway: ∂ CBD unifies three semantics into coherent framework



Practical impact

∂ CBDs provide a path from causal block diagrams to modern differentiable programming while retaining semantics for modular composition and verification.

Current scope

- Algebraic loops assumed absent or handled by delay / fixed-point blocks
- Residual contracts support system-level certificates only in special cases
- Scalability depends on execution graph, verification backend, and optimizer

Future directions

- Explicit algebraic-loop semantics and implicit differentiation
- Richer contract refinement and compositional proof rules
- Large-scale CPS case studies and toolchain integration

Long-term vision: compositional, certifiable scientific machine learning (SciML) for CPS.

Thank you

Questions?

∂ CBDs: composable diagrams, differentiable execution, contract-aware learning

Backup: A–G serial compatibility

For serial connection $\sigma_{1,2}$, upstream guarantees must supply signals admissible to downstream assumptions:

$$\sigma_{1,2}(\pi_{Y_1^\sigma}(\{(U_1, Y_1) \in \mathcal{G}_1 \mid U_1 \in \mathcal{A}_1\})) \subseteq \pi_{\sigma_{1,2}^{\text{in}}}(\mathcal{A}_2).$$

Interpretation

If block 1 promises an output under its assumptions, that output must satisfy the assumptions required by block 2.

Backup: implicit differentiation for solver blocks

For a fixed-point block

$$Y^* = \Phi(Y^*, U),$$

assuming $I - \partial_Y \Phi$ is nonsingular,

$$(I - \partial_Y \Phi) \delta Y^* = \partial_U \Phi \delta U, \quad J_{Y^*, U} = (I - \partial_Y \Phi)^{-1} \partial_U \Phi.$$

Message

Implicit solver blocks can expose JVP/VJP through linear solves rather than unrolling every iteration.

Backup: differentiable optimization layers

For parametric optimization

$$Y^*(U) = \arg \min_Y \ell(Y, U) \quad \text{s.t.} \quad h(Y, U) = 0,$$

KKT conditions define

$$K(Y^*, \lambda^*, U) = 0.$$

Under regularity,

$$\partial_{(Y, \lambda)} K \begin{bmatrix} \delta Y^* \\ \delta \lambda^* \end{bmatrix} + \partial_U K \delta U = 0.$$

Message

Optimization layers are valid ∂ CBD blocks when they expose differentiable solution maps.

Backup: speaker narrative in one slide

1. CPS engineers already use block diagrams, but learning and verification live elsewhere.
2. ∂ CBDs add contracts and derivative semantics directly to blocks.
3. Compositions preserve causality and contract structure.
4. Finite-horizon executions compile to AD-ready DAGs.
5. Residual contracts become differentiable objectives or constraints.
6. Examples show exact and trajectory-level stability certificates inside one optimization pipeline.